МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Київський національний університет будівництва і архітектури
Кафедра кібербезпеки та комп`ютерної інженерії

ЗВІТ

з лабораторної роботи № 6

Логування, моніторинг та rollback у ROS 2 системі

Виконав: студент групи БІКСм-26
Тарасов Максим

Перевірив: Маринський К.В

Київ – 2026

1. Опис реалізації логування

Метою роботи було навчитися реалізовувати логування в ROS 2, здійснювати моніторинг стану системи за допомогою сигналу активності (heartbeat), виявляти помилки та повертати систему до стабільної версії за допомогою системи контролю версій Git (rollback). Роботу виконано у віртуальній машині з Ubuntu та ROS 2 дистрибутива Lyrical, у робочому просторі ros2_ws із нодами попередніх лабораторних робіт.

Спочатку підготовлено середовище: підключено ROS 2 та робочий простір командою source, виконано ros2 node list і ros2 topic list. Жодна нода не була запущена, тому список нод порожній, а у списку топіків є лише службові /parameter_events і /rosout (рисунок 1).

```
source /opt/ros/lyrical/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 node list
ros2 topic list
```

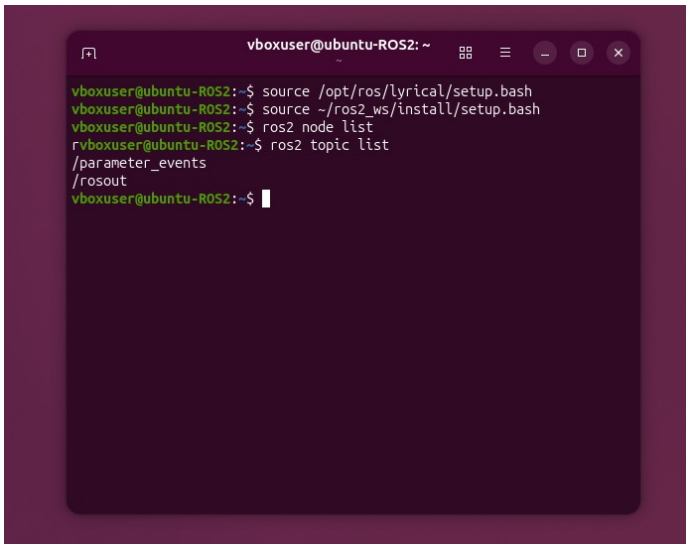


Рисунок 1 – Підготовка середовища: перевірка нод і топіків

Логування в ROS 2 виконується через логер ноди self.get_logger(), який має кілька рівнів важливості. У роботі використано три рівні, що відповідають завданню; їх призначення та приклади з нод системи наведено в таблиці 1. Кожне повідомлення виводиться в консоль у форматі [РІВЕНЬ] [мітка часу] [назва ноди]: текст.

Таблиця 1 – Рівні логування, використані в системі

Рівень	**Метод**	**Призначення**	**Приклад у системі**
INFO	get_logger().info()	штатна робота: запуск ноди, отримання даних	Motion Node: «Отримано: linear.x=0.50, angular.z=0.00»; heartbeat_node: «Heartbeat Node запущено»
WARN	get_logger().warning()	нештатна ситуація, система продовжує працювати	Motion Node: «Швидкість обмежено! …»; Sensor Node: «УВАГА! Перевищено поріг: …»
ERROR	get_logger().error()	помилка, що перешкоджає виконанню дії	Playback Node: «Файл не знайдено: …»

До Motion Node додано логування отримання даних: якщо швидкість перевищує допустимі межі й була обмежена, виводиться повідомлення рівня WARN із підсумковими значеннями, інакше – повідомлення рівня INFO із отриманими значеннями (рисунок 2). Запуск ноди також супроводжується повідомленням INFO («Motion Node запущено. Чекаю команди на /cmd_vel...»).

```
# Логування
if limited:
    self.get_logger().warning(
        f'Швидкість обмежено! '
        f'linear.x={linear_x:.2f}, angular.z={angular_z:.2f}'
    )
else:
    self.get_logger().info(
        f'Отримано: linear.x={linear_x:.2f}, angular.z={angular_z:.2f}'
    )
```

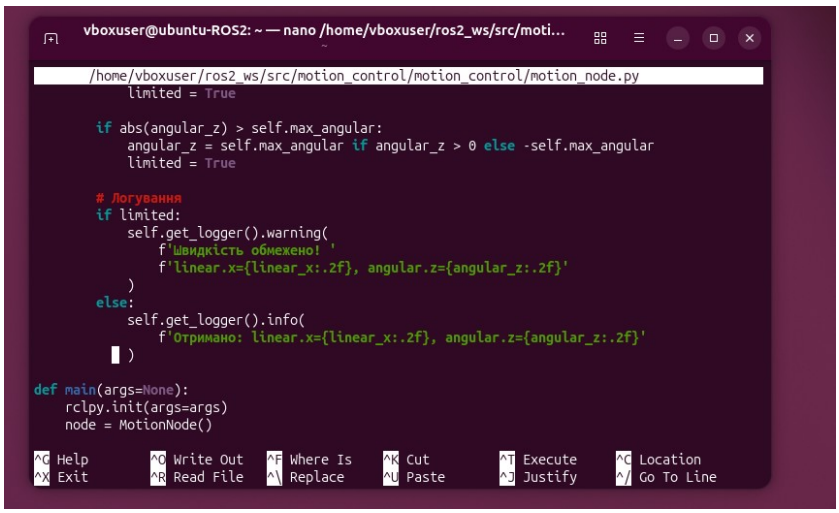


Рисунок 2 – Логування отримання даних у Motion Node (motion_node.py)

Таким чином, логуванням охоплено запуск ноди (INFO), отримання даних (INFO або WARN залежно від значень) та помилкові ситуації (WARN і ERROR), що дає змогу за журналом визначити стан кожної ноди без переривання її роботи.

2. Реалізація heartbeat

Heartbeat – це періодичний сигнал, за яким інші частини системи визнають, що нода жива. Для нього створено пакет system_monitor типу ament_python із залежностями rclpy і std_msgs (рисунок 3):

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python system_monitor \
    --dependencies rclpy std_msgs
```

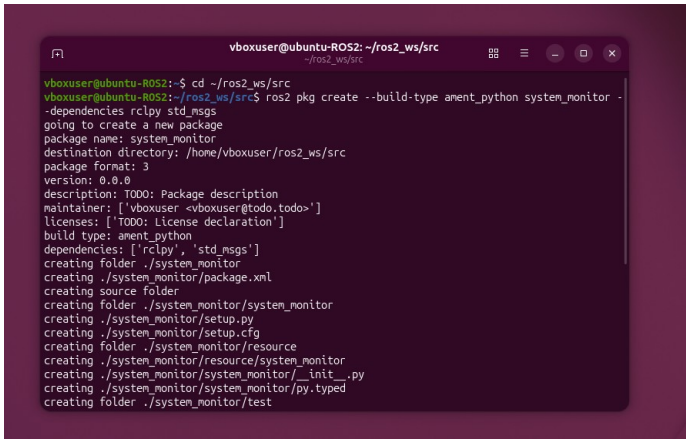


Рисунок 3 – Створення пакета system_monitor

У файлі setup.py до розділу entry_points додано точку входу heartbeat_node, яка вказує на функцію main модуля system_monitor.heartbeat_node (рисунок 4):

```
entry_points={
    'console_scripts': [
        'heartbeat_node = system_monitor.heartbeat_node:main',
    ],
},
```

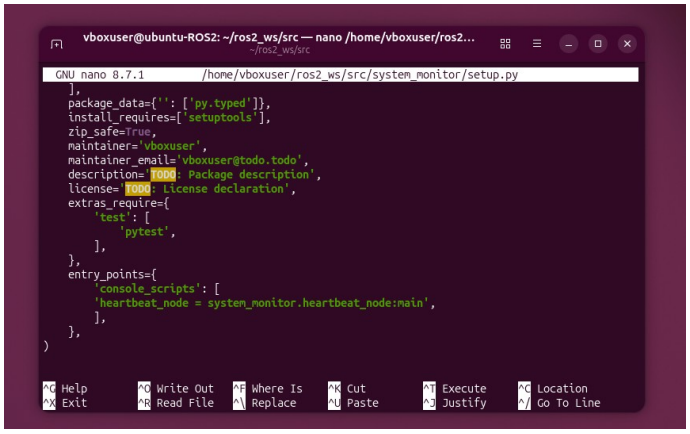


Рисунок 4 – Реєстрація точки входу heartbeat_node у файлі setup.py

Файлу ноди надано права на виконання командою chmod +x, після чого пакет зібрано командою colcon build --packages-select system_monitor. Збірка завершилась успішно за 2.79 с (загальний час 3.40 с), робочий простір підключено через source install/setup.bash (рисунок 5).

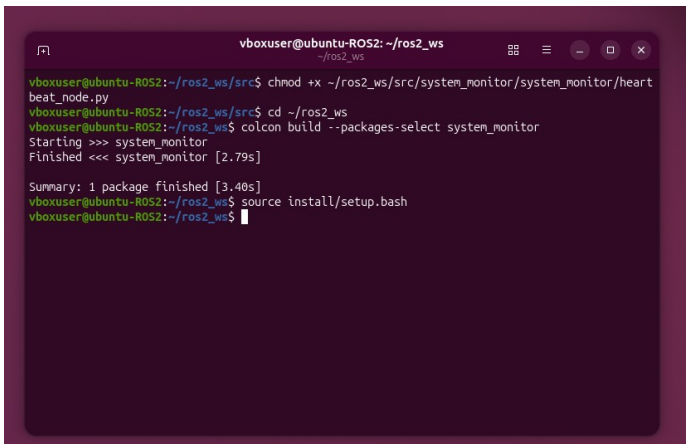


Рисунок 5 – Збірка пакета system_monitor

Нода heartbeat_node за таймером з періодом 1 с публікує в топік /heartbeat повідомлення типу std_msgs/msg/String, що містить ознаку «alive», лічильник відправлених повідомлень count та поточний час Unix. Те саме повідомлення виводиться в журнал рівня INFO. Основні фрагменти коду:

```
self.publisher_ = self.create_publisher(String, '/heartbeat', 10)
self.timer = self.create_timer(1.0, self.timer_callback)  # кожну секунду
self.counter = 0

def timer_callback(self):
    msg = String()
    msg.data = f'alive | count={self.counter} | time={time.time():.0f}'
    self.publisher_.publish(msg)
    self.get_logger().info(f'Heartbeat: {msg.data}')
    self.counter += 1
```

Нода запущена командою ros2 run system_monitor heartbeat_node. Вона вивела повідомлення про запуск і далі щосекунди публікувала сигнал із лічильником 0, 1, 2, … (рисунок 6). Повний код ноди наведено у файлі heartbeat_node.py, що додається до звіту.

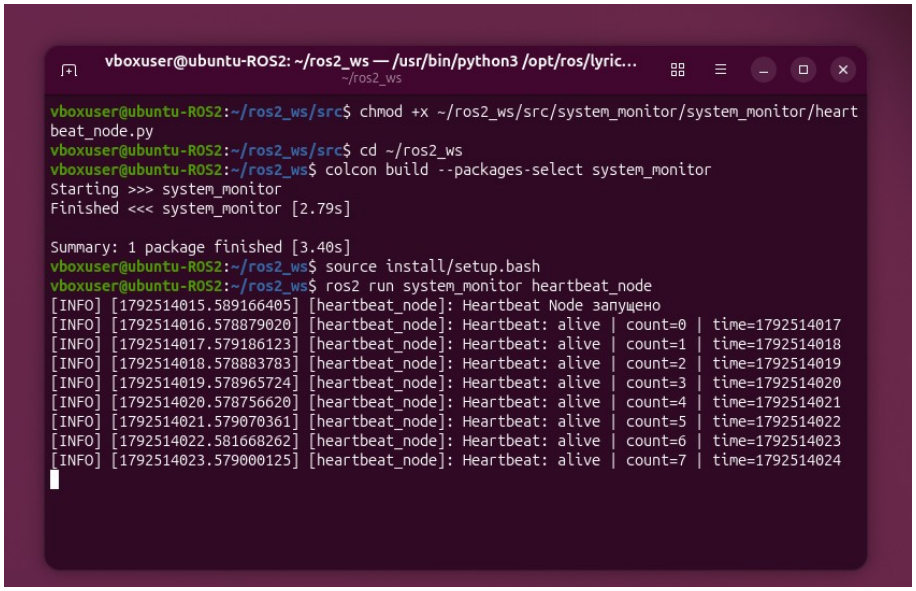


Рисунок 6 – Запуск heartbeat_node: щосекундні повідомлення в журналі

3. Опис моніторингу

Моніторинг стану системи виконується за допомогою топіка /heartbeat. Для спостереження за ним у другому терміналі використовується команда:

```
ros2 topic echo /heartbeat
```

За повідомленнями топіка (а також за журналом ноди, рисунок 6) оператор оцінює три показники, перелічені в завданні:

- чи працює нода – поки нода жива, повідомлення з’являються в топіку; відсутність нових повідомлень понад кілька секунд означає, що нода зупинилась або зависла (також її відсутність видно в ros2 node list);
- чи приходять дані – поле count збільшується на одиницю з кожним повідомленням, тому пропуск значення вказує на втрату повідомлення; поле time дозволяє визначити момент останнього сигналу;
- чи стабільна система – інтервали між сигналами мають бути близькими до 1 с, а в журналах нод не повинно бути повідомлень рівнів WARN та ERROR.

Для зафіксованого запуску (рисунок 6) показники такі: лічильник пройшов значення від 0 до 7 без пропусків, а інтервали між журнальними мітками часу склали 1.000, 1.000, 1.000, 1.000, 1.000, 1.003 та 0.997 с, тобто середній період дорівнює 1.0 с із відхиленням не більше 3 мс (таблиця 2). Це свідчить про стабільну роботу ноди та таймера.

Таблиця 2 – Оцінка стабільності за сигналом heartbeat

Показник	**Очікуване значення**	**Спостережуване значення**
Період публікації	1.0 с	1.0 с (відхилення ≤ 0.003 с)
Лічильник count	0, 1, 2, … без пропусків	0 – 7 без пропусків
Рівні в журналі ноди	лише INFO	лише INFO (WARN/ERROR відсутні)

4. Приклад rollback

Для версіонування робочого простору використано Git. Оскільки на віртуальній машині Git не був встановлений (команда git init повернула повідомлення «Command 'git' not found»), його встановлено командою sudo apt install git (рисунок 7).

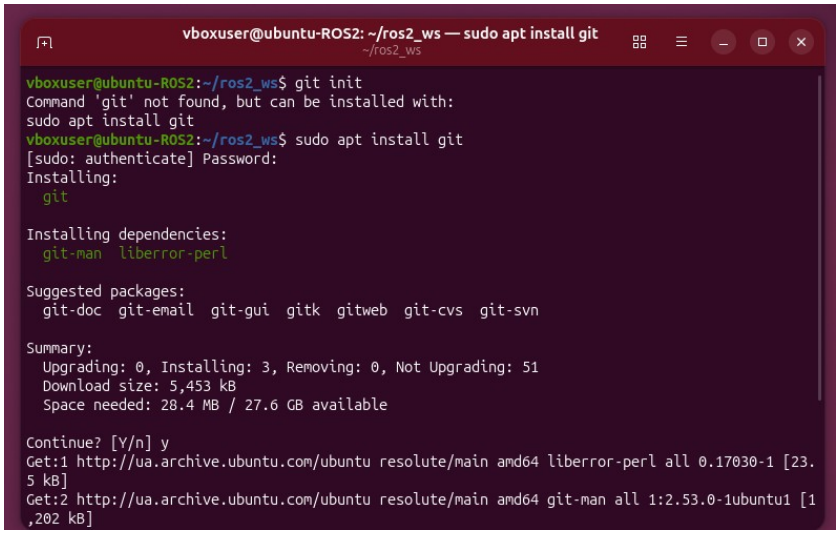


Рисунок 7 – Встановлення Git

У каталозі ~/ros2_ws ініціалізовано репозиторій, задано ім’я та адресу автора коміту, додано всі файли й створено коміт стабільної версії (рисунок 8):

```
cd ~/ros2_ws
git init
git config --global user.email "tarasov_mi-22@knuba.edu.ua"
git config --global user.name "MaksymT"
git add .
git commit -m "Stable version - Lab6 before changes"
```

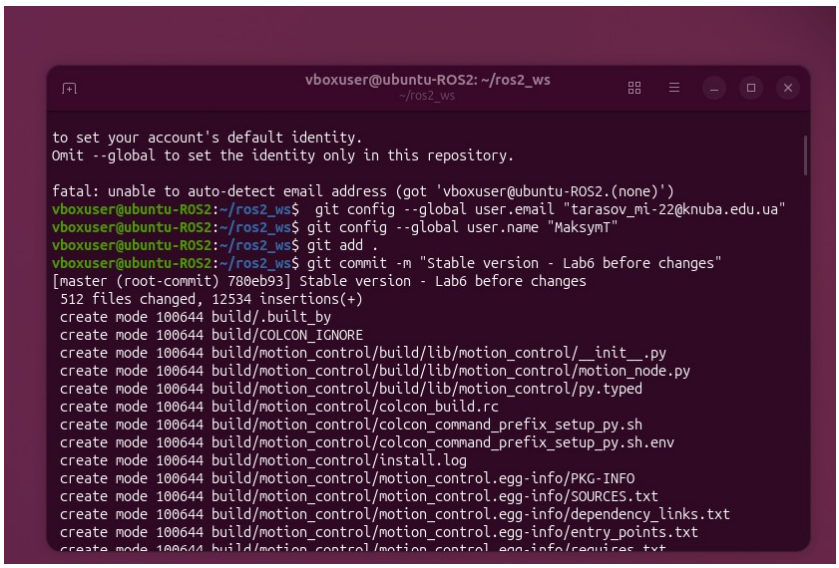


Рисунок 8 – Коміт стабільної версії (780eb93)

Далі у файл motion_node.py умисно внесено помилку: після імпортів додано рядок print(32j4i32io), який не є коректним виразом Python (рисунок 9). Нода з таким файлом не може бути запущена, оскільки інтерпретатор повідомляє про синтаксичну помилку ще до виконання коду. Змінений файл також зафіксовано в окремому коміті «Broken version - introduced bug» (рисунок 10); історія репозиторію після цього налічує два коміти: 794ebe5 (з помилкою) та 780eb93 (стабільний).

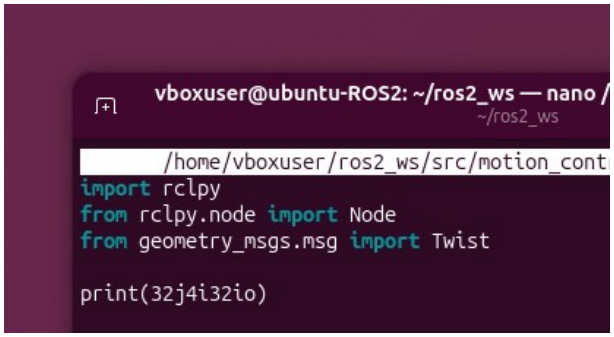


Рисунок 9 – Помилка, внесена у motion_node.py

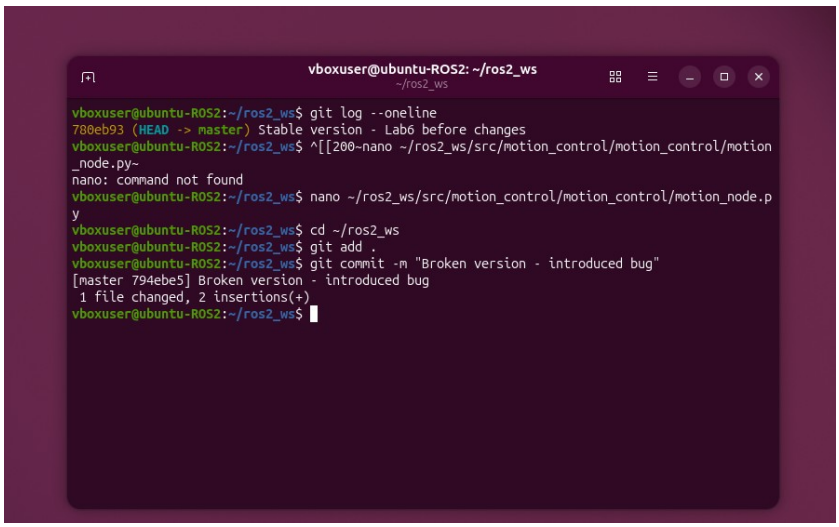


Рисунок 10 – Коміт версії з помилкою (794ebe5) та перегляд історії

Rollback виконано командою git checkout HEAD~1, яка перемістила робочу копію на коміт, що передує поточному, тобто на стабільну версію 780eb93 (рисунок 11). Git повідомив, що репозиторій перебуває у стані «detached HEAD»: це нормально для тимчасового повернення до старої версії, а повернутись до останнього коміту можна командою git switch -.

```
git log --oneline
git checkout HEAD~1
```

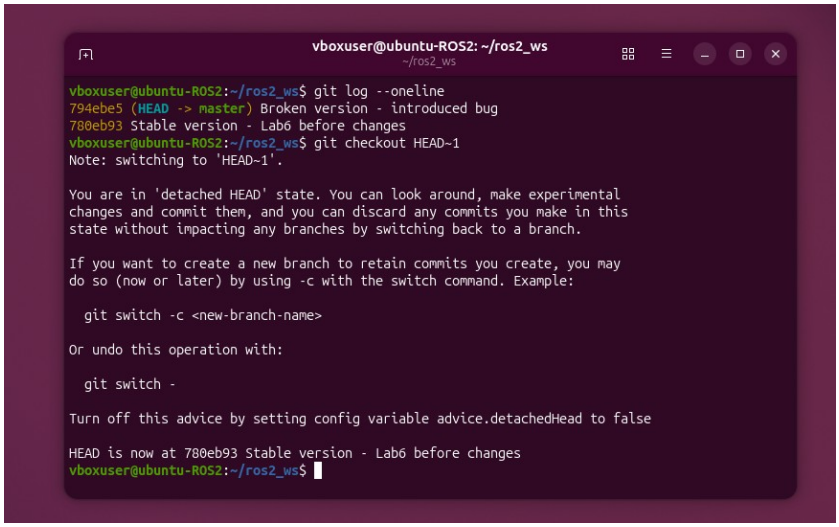


Рисунок 11 – Rollback: git checkout HEAD~1

Після відновлення вихідного коду робочий простір перезібрано командою colcon build. Усі шість пакетів (motion_control, my_py_pkg, robot_gui, scenario_recorder, sensor_node, system_monitor) зібрано без помилок за 5.70 с, після чого виконано source install/setup.bash (рисунок 12). Успішна збірка всіх пакетів, зокрема motion_control, підтверджує, що після rollback у файлі motion_node.py більше немає внесеної помилки.

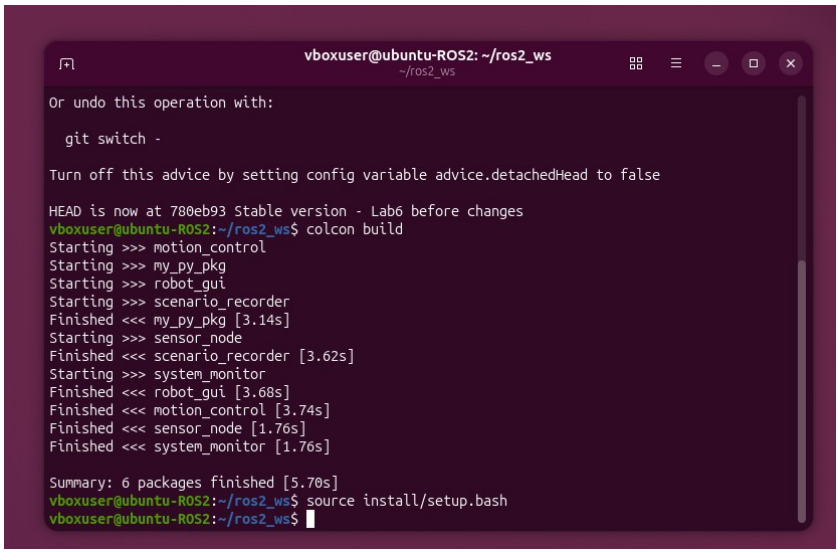


Рисунок 12 – Перезбирання workspace після rollback

Слід зазначити, що команда git add . додала до першого коміту й каталоги build/, install/ та log/ (було зафіксовано 512 файлів). Для чистішої історії в таких проєктах до репозиторію зазвичай додають файл .gitignore із цими каталогами, щоб зберігати лише вихідний код.

5. Висновки

У ході лабораторної роботи реалізовано логування у ROS 2 із використанням рівнів INFO, WARN та ERROR: у Motion Node додано повідомлення про отримані й обмежені команди, а в системі в цілому логуванням охоплено запуск нод, отримання даних і помилки. Створено пакет system_monitor із нодою heartbeat_node, яка щосекунди публікує в топік /heartbeat сигнал активності типу std_msgs/msg/String з лічильником і часом.

За цим сигналом організовано моніторинг: перевіряється наявність повідомлень, послідовність лічильника та стабільність інтервалу. У

тестовому запуску лічильник йшов без пропусків, а період становив 1.0 с з відхиленням не більше 3 мс. За допомогою Git створено коміт стабільної версії, внесено помилку, зафіксовано її в окремому коміті та виконано rollback командою git checkout HEAD~1; після перезбирання всі шість пакетів зібрано без помилок.

Отримані навички дозволяють швидко виявляти несправності за журналами й heartbeat та безпечно повертатися до робочої версії коду після невдалих змін. Для повнішого захисту системи можна додати ноду-спостерігача, яка підписується на /heartbeat і виводить повідомлення рівня ERROR, якщо сигнал не надходив понад певний час, а також використовувати гілки Git замість стану detached HEAD.